DOCUMENTACIÓN DE PRUEBAS

PROYECTO: Traduce Señas

INTEGRANTES:
Luis Alejandro Duarte Aldana
Juan Mauricio Suaza Solorzano
Juan Camilo Fierro Esquivel

SERVICIO NACIONAL DE APRENDIZAJE – SENA
Análisis y Desarrollo De Software

2026

Tabla de contenido

Introducción 3
Objetivos de las pruebas 3
Estrategia de pruebas 3
Plan de pruebas 4
Elementos a probar 4
Elementos fuera de alcance de esta iteración 4
Entorno de pruebas 4
Criterios de aceptación y suspensión 5
Roles 5
Resultados de las pruebas automatizadas 5
Casos de prueba funcionales 7
Pruebas no funcionales 9
Pruebas del reconocimiento de señas 9
Defectos encontrados 10
Conclusiones 10
Referencias 12

Introducción

La documentación de pruebas reúne la estrategia, el plan, los casos de prueba y los resultados obtenidos sobre el software Traduce Señas. Su propósito es demostrar con evidencia que el sistema cumple los requisitos del informe de especificación y dejar identificados los aspectos que todavía no se han verificado.

La estructura del documento toma como referencia la norma ISO/IEC/IEEE 29119-3 sobre documentación de pruebas (International Organization for Standardization [ISO], 2013) y el modelo de calidad ISO/IEC 25010 (ISO, 2011).

Objetivos de las pruebas

- Verificar que cada microservicio cumple su lógica de negocio y sus reglas de seguridad.
- Comprobar que los servicios se integran correctamente a través del gateway y de los eventos.
- Validar los requisitos funcionales críticos (autenticación, traducción, historial, alfabeto, estadísticas).
- Evaluar los requisitos no funcionales de accesibilidad, rendimiento y seguridad.

Estrategia de pruebas

Se aplica la pirámide de pruebas: muchas pruebas unitarias rápidas en la base, menos pruebas de integración y contrato en el medio, y pocas pruebas de extremo a extremo en la cima.

Tabla 1
Niveles de prueba

Nivel	Objetivo	Herramienta	Meta
Unitarias	Lógica de dominio y casos de uso en aislamiento, con repositorios falsos	node:test (backend); Jest + jest-expo (frontend)	≥ 80 % de líneas; ≥ 90 % en domain/
Integración	Adaptadores con PostgreSQL, MongoDB y RabbitMQ reales	node:test + Docker Compose	Camino feliz y 2–3 errores por adaptador
Contrato	Las respuestas HTTP cumplen el contrato OpenAPI	Contratos YAML de 07-api	100 % de rutas públicas

Nivel	Objetivo	Herramienta	Meta
Extremo a extremo	Flujos completos del usuario a través del gateway	Playwright en CI contra el backend en Docker Compose; cliente HTTP / prueba manual	Flujos críticos
Aceptación	Validación con usuarios reales (personas sordas y oyentes)	Prueba piloto guiada	Casos de uso CU-01 a CU-06
Accesibilidad	Contraste, tamaño de objetivos y lectores de pantalla	Accessibility Scanner, TalkBack, revisión manual	WCAG 2.1 AA
Rendimiento	Latencia y rendimiento de los servicios críticos	k6	P95 < 300 ms con 50 peticiones/s

Plan de pruebas

Elementos a probar

- api-gateway y los cinco microservicios (backend versión 1.4.0, rama main, commit a962982).
- Paquete compartido @traduce/shared.
- Aplicación cliente en Android y en navegador de escritorio, con pruebas unitarias de su lógica: validaciones, cliente de la API, flujos de autenticación y reconocimiento (frontend versión 1.3.1, rama main, commit a5e231d).

Elementos fuera de alcance de esta iteración

- Pruebas de carga sobre el servidor de producción (traducesenas.tech), para no afectar a los usuarios.
- Publicación en tiendas de aplicaciones.
- Precisión del reconocimiento de las 20 palabras del conjunto LSC-54 (pendiente de la medición descrita en la sección de reconocimiento).

Entorno de pruebas

Tabla 2
Entorno utilizado

Elemento	Detalle
Equipo	Computador con Windows 11, Node.js 24 y Docker Desktop
Backend	Docker Compose: gateway, 5 servicios, 5 PostgreSQL 16, MongoDB 7, RabbitMQ 3
Cliente	Expo web en Chrome; Expo Go en Android
Datos	Usuarios y señas de prueba creados por las propias pruebas; ningún dato real

Criterios de aceptación y suspensión

- **Aceptación:** 100 % de las pruebas automatizadas en verde y todos los casos críticos aprobados.
- **Suspensión:** se detiene la ejecución si falla la compilación o si una prueba crítica de seguridad falla.
- **Reanudación:** tras corregir el defecto y volver a ejecutar la suite completa.

Roles

Tabla 3
Responsables de las pruebas

Rol	Responsable	Tareas
Líder de pruebas	Juan Mauricio Suaza Solórzano	Plan, pruebas de backend, integración y reporte
Probador de interfaz	Luis Alejandro Duarte Aldana	Pruebas de pantallas y accesibilidad
Probador funcional	Juan Camilo Fierro Esquivel	Casos de aceptación y prueba piloto

Resultados de las pruebas automatizadas

Las suites automatizadas se ejecutaron con «npm test» en cada repositorio. En el backend, el comando compila el paquete compartido y corre las pruebas de todos los servicios con el ejecutor nativo de Node.js; en el frontend, corre Jest con el preset de Expo (jest-expo). El resultado vigente, de la ejecución del 4 de noviembre de 2026, es: en el backend, 918 pruebas aprobadas, 0 fallidas y 13 omitidas por requerir PostgreSQL en el equipo local; en el frontend, 469 pruebas en 46 suites, todas aprobadas. La primera corrida del frontend tuvo un timeout de 5 s en AchievementUnlockAlerts.test.tsx con la máquina cargada; pasó al ampliar el tiempo de espera a 30 s («--testTimeout=30000»). La tabla 4 conserva la ejecución del 16 de octubre de 2026 como referencia histórica:

Tabla 4
Resumen de la ejecución automatizada del 16 de octubre de 2026

Indicador	Backend	Frontend
Pruebas ejecutadas	683	79
Suites (grupos)	141	8
Aprobadas	681	79
Fallidas	0	0
Omitidas / canceladas	2 / 0	0 / 0

Indicador	Backend	Frontend
Duración total	22 segundos	3 segundos

Nota. Ejecución del 16 de octubre de 2026. Backend: rama develop, commit 2993301; las 2 pruebas omitidas necesitan una base PostgreSQL de prueba (variable PROFILE_TEST_DATABASE_URL). Frontend: rama chore/frontend-unit-tests, commit 6219c0d.

Tabla 5

Cobertura de las pruebas por componente

Componente	Archivos de prueba	Aspectos verificados	Cobertura de líneas
@traduce/shared	7	Proveedor JWT, authMiddleware, requireRole, requirePermission, manejador de errores, configuración, validadores de eventos, reintento exponencial, bus en memoria y RabbitMQ	99,5 %
api-gateway	3	Enrutamiento por prefijo, CORS con lista blanca, configuración, rutas públicas, límite de peticiones	100 %
iam-service	33	Registro con verificación de correo, inicio de sesión según estado y con Google o Facebook, recuperación de contraseña, JWT con roles y permisos, cambio y borrado de cuenta (también con código por correo), envío de eventos con outbox, plantillas de correo	89,3 %
recognition-service	17	Traducciones, historial, plantillas de señas (solo ADMIN), muestras, consumo de eventos de lexicon con contrato y rechazo de escrituras de cuentas eliminadas	86,0 %
lexicon-service	5	Catálogo, alfabeto, búsqueda, categorías, publicación y retiro de señas	83,5 %
analytics-service	6	Registro de eventos, reportes, manejadores de eventos y borrado atómico de los datos de una cuenta	95,1 %
profile-service	27	Perfil, preferencias con permisos y creación del perfil faltante, notificaciones, logros con actualizaciones simultáneas, borrado de cuenta y recuperación de perfiles	83,5 %
Frontend (aplicación cliente)	46 Jest (más 3 E2E con Playwright)	Validación de correo y contraseña, cliente de la API (errores, token enviado solo a la API, aviso único de sesión caducada), flujos de confirmar correo y recuperar contraseña, contador de reenvío, normalización de la mano y clasificador KNN	4,5 %

Nota. Las cifras de cobertura de líneas se midieron el 16 de octubre de 2026 con la cobertura integrada de node:test en el backend (total 88,5 %) y con «jest --coverage» en el frontend; esa corrida es anterior a las pruebas descritas abajo y no se ha repetido. La columna de archivos de prueba es del 4 de noviembre de 2026 y cuenta los archivos de cada componente en la rama main.

Pruebas añadidas después de esa corrida. En el frontend (versión vigente 1.3.1) se agregaron pruebas de Jest para los servicios, los formularios de autenticación, los clasificadores de visión y los almacenes de plantillas, y una suite extremo a extremo con Playwright (carpeta e2e/: auth, alphabet e history). Esa suite se ejecuta con «npm run e2e»

sobre la web exportada, contra el backend levantado con Docker Compose, con un solo worker (el gateway limita los inicios de sesión por minuto), ancho de celular y cámara falsa del navegador; en CI corre contra el backend en compose. En el backend (profile-service) se agregaron pruebas de retos diarios (dominio, casos de uso, API y PostgreSQL real), de las medallas por puntos de reto y del envío de correos a los administradores.

Casos de prueba funcionales

Los casos siguientes verifican los requisitos críticos. El estado «Aprobado» indica que el caso fue ejecutado con éxito; «Pendiente» indica que aún no se ha ejecutado en el entorno correspondiente.

Tabla 6
Casos de prueba funcionales

ID	Requisito	Caso	Datos de entrada	Resultado esperado	Estado
CP-01	FR-001, FR-003	Registro con verificación de correo	Correo nuevo, datos válidos, términos aceptados	Cuenta INACTIVE; código enviado; activa tras verificar	Aprobado (automatizada y E2E)
CP-02	FR-002	Registro sin aceptar términos	terms_accepted = false	400; no se crea la cuenta	Pendiente (sin prueba automatizada específica)
CP-03	FR-005	Inicio de sesión válido	Credenciales correctas, cuenta ACTIVE	200 con token JWT que contiene roles	Aprobado (automatizada y E2E)
CP-04	FR-005	Inicio de sesión con cuenta INACTIVE o BLOCKED	Cuenta sin verificar o bloqueada	Acceso denegado con mensaje claro	Aprobado (automatizada)
CP-05	FR-004, FR-010	Recuperación con código vencido o usado	Código expirado / ya usado	Rechazo; se cuentan los intentos	Aprobado (automatizada)
CP-06	RNF4	Límite de peticiones en /api/auth	Ráfaga de solicitudes desde una IP	429 Demasiadas peticiones	Aprobado (automatizada)
CP-07	FR-103	Usuario sin rol ADMIN importa plantillas	Token con rol USER	403 Prohibido	Aprobado (automatizada)

ID	Requisito	Caso	Datos de entrada	Resultado esperado	Estado
CP-08	FR-401, FR-406	Traducción guardada en historial	Seña reconocida, usuario autenticado	201; aparece en el historial	Aprobado (E2E con Docker)
CP-09	FR-802	Logro «Primera traducción»	Primera traducción de un usuario	Logro desbloqueado por evento TranslationProduced	Aprobado (E2E con Docker)
CP-10	FR-701	Evento de uso registrado	Traducción guardada	Evento en analytics-service	Aprobado (E2E con Docker)
CP-11	FR-405	Seña no reconocida	Gesto fuera del vocabulario	Aviso visual; no se muestra palabra	Pendiente (prueba manual en dispositivo)
CP-12	FR-304, FR-310	Consulta del alfabeto	GET /api/lexicon/alphabet	Solo letras de la LSC en estado ACTIVE	Aprobado (automatizada)
CP-13	FR-406	Historial privado	Usuario A consulta historial de B	Solo ve sus propias traducciones	Pendiente (prueba manual)
CP-14	FR-806, FR-807	Reto diario completado	Usuario que guarda tantas traducciones como la meta del día	Progreso hasta la meta; recompensa guardada una sola vez; aviso CHALLENGE_COMPLETED	Aprobado (automatizada, ejecución del 4 de noviembre de 2026)
CP-15	FR-808	Medalla por puntos de reto	Usuario que acumula puntos de reto	GET /api/profile/challenges/rewards/summary devuelve el total; la medalla se otorga una vez	Aprobado (automatizada, ejecución del 4 de noviembre de 2026)
CP-16	FR-409, FR-707	Historial por sesiones y URL propia	Abrir el historial por su URL con varias traducciones guardadas	Una tarjeta por sesión; la pantalla se recarga sin perder la ruta	Pendiente (el E2E de historial cubre guardar y borrar una traducción, pero no las sesiones ni la URL propia)
CP-17	FR-410	Retención de 30 días	Traducción con más de 30 días	La traducción deja de aparecer en el historial	Pendiente (prueba manual)
CP-18	FR-014	Eliminar cuenta	Usuario autenticado	Datos borrados en una transacción y eventos de uso anonimizados	Aprobado (automatizada)
CP-19	FR-602	Muestra sin consentimiento	Muestra sin campo de consentimiento	400; la muestra no se guarda	Aprobado (automatizada)

ID	Requisito	Caso	Datos de entrada	Resultado esperado	Estado
CP-20	FR-706	Estadísticas con rol USER	Token sin permiso	403 Prohibido	Aprobado (automatizada)

Nota. La prueba de extremo a extremo con Docker (registro → inicio de sesión → traducción → logro y evento de uso) se ejecutó el 5 de octubre de 2026.

Pruebas no funcionales

Tabla 7
Estado de las pruebas no funcionales

Atributo	Prueba	Criterio	Estado
Accesibilidad	Contraste de colores del sistema de diseño	≥ 4,5:1	Aprobado: tokens ajustados en el sistema de diseño
Accesibilidad	Navegación con TalkBack en flujos principales	Flujos completos sin bloqueo	Pendiente
Seguridad	Secreto JWT obligatorio de 32+ caracteres	El servicio no arranca sin él	Aprobado (automatizada)
Seguridad	Bases de datos y broker no expuestos a la red	Puertos solo en 127.0.0.1	Aprobado (revisión de docker-compose)
Rendimiento	Latencia P95 de login, traducciones y léxico	< 300 ms con 50 peticiones/s	Pendiente (k6)
Rendimiento	Cuadros por segundo en la captura	≥ 15 FPS en gama media	Pendiente
Tolerancia a fallos	Caída y regreso de RabbitMQ	El bus se reconecta y reintenta	Aprobado (automatizada)

Pruebas del reconocimiento de señas

La precisión del reconocimiento no se mide con pruebas unitarias sino con un protocolo de evaluación: se toman muestras de personas que no participaron en la construcción de las plantillas, se aplica el mismo método de la aplicación (normalización y Dynamic Time Warping) y se calcula, por cada seña, el porcentaje de aciertos, el de palabras equivocadas y las señas con las que se confunde.

Para las 20 palabras nuevas se usa el conjunto público LSC-54 (Mora-Zarate et al., 2025), separando a los firmantes de entrenamiento y de prueba. La meta de aceptación es una precisión de al menos 85 % antes de publicar las plantillas. Esta medición está pendiente porque la descarga de los datos se detuvo por las restricciones del servidor de origen.

Defectos encontrados

Tabla 8
Registro de defectos

ID	Descripción	Severidad	Estado
DEF-01	El descargador del conjunto LSC-54 podía omitir videos completos sin avisar	Alta	Corregido
DEF-02	Contador de logros no atómico: se perdían incrementos con traducciones simultáneas	Media	Corregido: actualización atómica con ON CONFLICT DO UPDATE
DEF-03	Nombre de usuario sin escapar en el HTML del correo de recuperación	Alta (seguridad)	Corregido: función escapeHtml
DEF-04	GET /api/profile/me fallaba si el perfil aún no existía	Media	Mitigado: el perfil faltante se crea al guardar las preferencias y hay un procedimiento de recuperación; la consulta sigue respondiendo 404 mientras falte
DEF-05	Prueba de contrato de eventos de lexicon fallaba (305 de 306)	Baja	Corregido: la prueba pasa en la suite actual
DEF-06	La letra L nunca se reconoce: su forma de mano coincide antes con la regla de la D o la G	Media	Abierto (encontrado con las pruebas unitarias del frontend)
DEF-07	Dos migraciones de iam-service con el mismo número (010) al integrar el borrado de cuenta	Media	Corregido: renumerada a 012 antes del despliegue
DEF-08	Una prueba de recognition-service respondía 500 por la nueva protección de cuentas eliminadas	Baja	Corregido: la prueba usa una conexión simulada

Nota. DEF-01 a DEF-05 provienen de la auditoría de código y seguridad de la fase 4 de la migración a microservicios; DEF-06 a DEF-08, de las pruebas unitarias del frontend y de la revisión de código del 16 de octubre de 2026.

Conclusiones

El backend pasa su suite automatizada: 918 pruebas aprobadas y ninguna fallida; 13 se omiten en el equipo local porque necesitan una base PostgreSQL. El frontend pasa sus 469 pruebas Jest (46 suites), y sus 3 pruebas de extremo a extremo con Playwright corren en CI. La cobertura de líneas se midió por última vez el 16 de octubre de 2026: 88,5 % en el backend, por encima de la meta de 80 %, y 4,5 % en el frontend, que todavía es baja. Jest no tiene un umbral de cobertura configurado, aunque el ADR-006 fija una puerta de 80 %. La integración continua ya corre en ambos repositorios. Los flujos críticos de extremo a extremo funcionan con todos los servicios desplegados en Docker. Quedan pendientes ampliar las pruebas del frontend a las pantallas, las pruebas de rendimiento con k6, las pruebas con lector de pantalla,

la medición de precisión del reconocimiento de las nuevas palabras y la prueba piloto con usuarios reales, que se programan para la siguiente iteración.

Referencias

International Organization for Standardization. (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models* (ISO/IEC 25010:2011). ISO.

International Organization for Standardization. (2013). *Software and systems engineering — Software testing — Part 3: Test documentation* (ISO/IEC/IEEE 29119-3:2013). ISO.

Mora-Zarate, J. E., Garzón-Castro, C. L., & Castellanos Rivillas, J. A. (2025). LSC-54: A landmark-based dataset for Colombian Sign Language. *Data in Brief*. https://doi.org/10.1016/j.dib.2025.112145

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9.ª ed.). McGraw-Hill Education.

Sakoe, H., & Chiba, S. (1978). Dynamic programming algorithm optimization for spoken word recognition. *IEEE Transactions on Acoustics, Speech, and Signal Processing, 26*(1), 43–49. https://doi.org/10.1109/TASSP.1978.1163055

Sommerville, I. (2016). *Software engineering* (10.ª ed.). Pearson.

World Wide Web Consortium. (2018). *Web Content Accessibility Guidelines (WCAG) 2.1*. W3C. https://www.w3.org/TR/WCAG21/